

A decorative graphic on the left side of the slide consisting of two overlapping parallelograms. The front one is blue and the back one is a light green color. They are positioned diagonally, with the blue one partially covering the green one.

The Blank OSL

Presented by: Team Vertex (Alan, Evan, Pouria, Shirley, Tony)



Team Vertex

- Team of 5 members
 - All on campus this term and next term
 - Diverse expertise and background
 - Shared vision of building AI-driven application
- Organization
 - Weekly standup
 - Github -> Gitlab
 - Discord



Advisor / Mentor

Ramani T.

Research Engineer

@ Google DeepMind



Recap of timeline



Pitch Presentation

Research MCP
Setup Dev Env
Brainstorm MCP Servers

Midterm Presentation

Upgrade to MCP Servers
User Confirmation Feature
Setup Claude Desktop

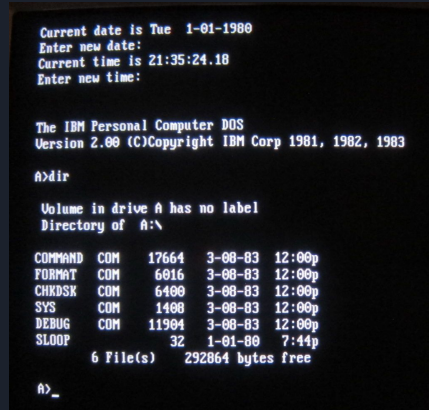
Final Presentation

Support Various Platforms
Continue Updating/Adding
Servers

Evolution of User Interface

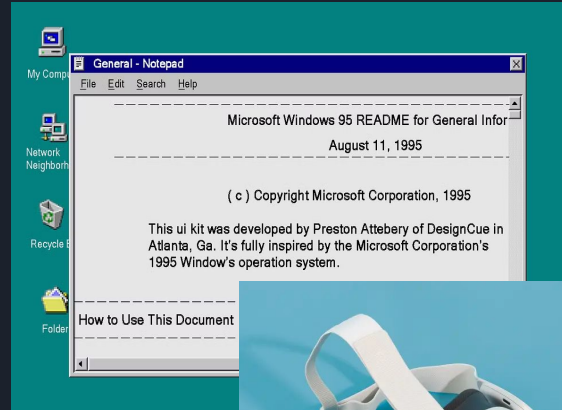
1970s - 1980s

Text-based user interface



1980s - 2007s

Graphical user interface

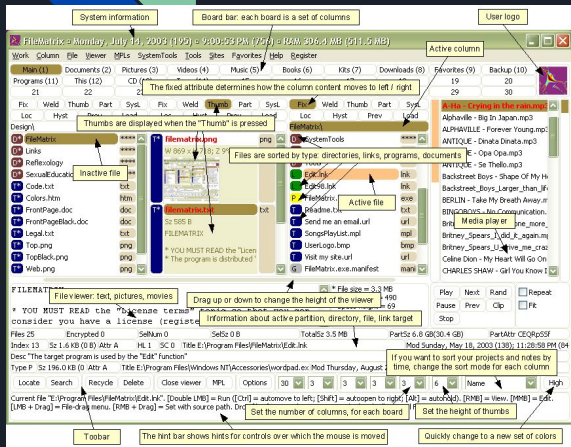


2007s - Present

Finger/Voice-based user interface



Our vision



Stupidly simple.



Stupidly intuitive.

How can you make money as a kid?

Google:



Bing:

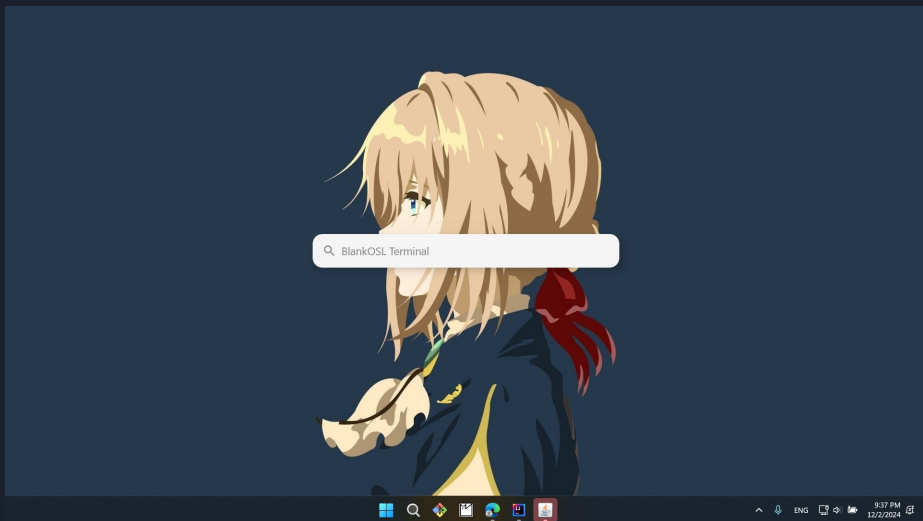
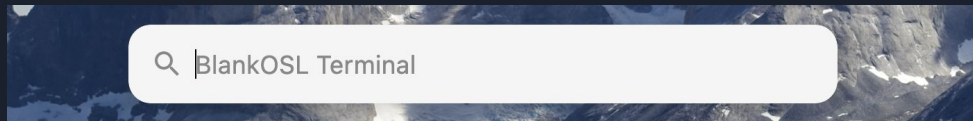


Stupidly smart.

Project Overview

The Blank OSL:

An Operating System Layer (OSL) utilizes LLMs to improve OS navigation.



- The UI of all UI.
- Minimalist UI design.
- Beginner friendly with minimum knowledge required.
- Fast-paced environment that requires quick solution.



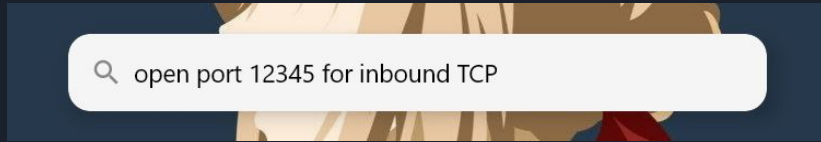
Recap: Target User / Use Cases

- **Manual Operation Work Automation**
 - Automate repetitive tasks (e.g., sorting emails, scheduling)
 - Integrate various apps into a single workflow
- **Paperwork Assistance (e.g., Visa Application)**
 - Translate and complete forms in unfamiliar languages
 - Gather necessary documents, fill out applications, and submit on your behalf

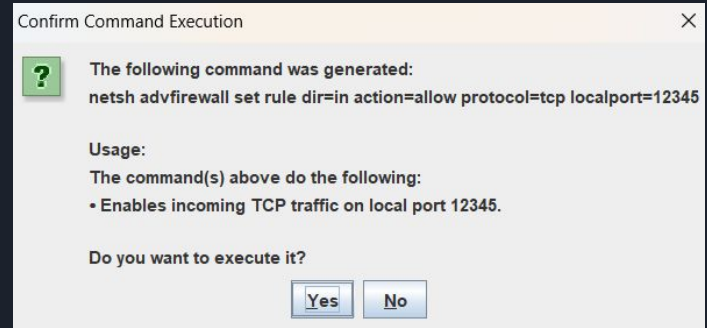
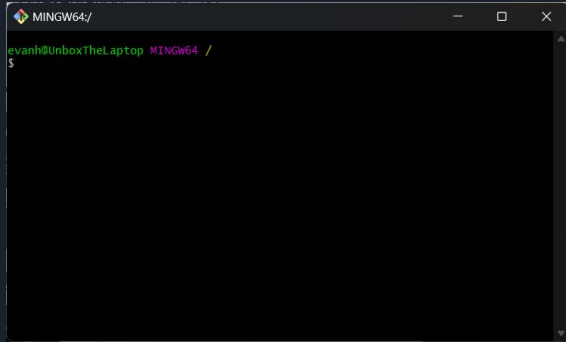
Recap: The bad old unreliable vibe prompting.

Terminal UI

LLM Clients



Gemini

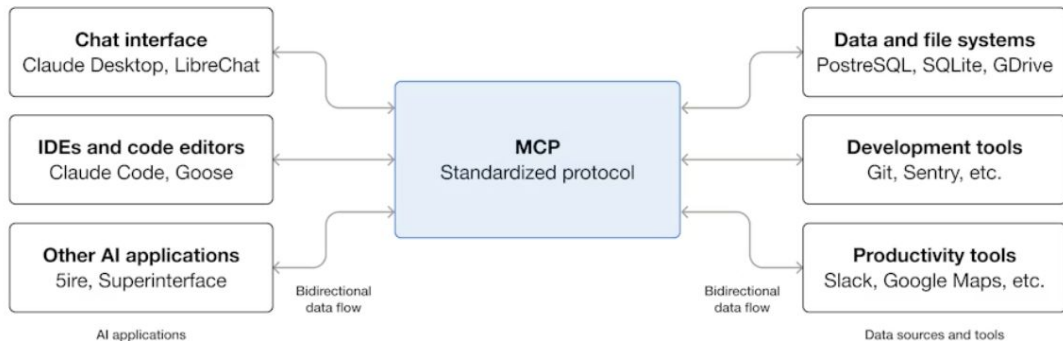


Command Executor

Confirmation

Pivot to Model Context Protocol (MCP)

Think of MCP like a **USB-C** port for AI applications. MCP provides a standardized way to connect AI applications to external systems.



Status



Core OS Functions

- OS File System
- Tasks Manager
- Contacts Manager



Communication & Files

- Sending & reading emails
- PDF, CSV, ZIP common file



Development & Integration

- Docker
- Dedicated HTTP Clients



Applications

- Weather
- Stocks
- ...

Implementation Challenges

Filesystem



- Sandbox Security / Unauthorized Write/Delete Files
- Directory Traversal

Email



- Credential Management / Third-Party API Access (Gmail)

PDF



- Complex Content Parsing (e.g., SVG, Charts)
- Conversion (HTML → PDF)

Other MCP Server



- API Key Exposure / Sensitive Data Leakage
- Rate Limit Failure / Accidental Throttling

Live Demo with Claude!





Command # 1

Hi I am Shirley! I'm filling out my insurance application.

Can you please check if the insurance agent Maxi has emailed the application form to me yet?



Command # 2

Please download the attached PDF, fill in my information, and save the completed form to my Downloads folder.



Command # 3

Great!

Can you please help me email the filled insurance form back to Maxi?

Current and Future Progress

- Upgrade to MCP ✓
- User Confirmation Feature ✓
- Various Feature Improvements ✓
- Standardized dev environment with Docker ✓
- Finding testing users ✓
- Customized MCP Client (SE491)
- Speech to Text Feature (SE491)

BlankOSL Terminal

s Claude wants to use Get stock price from stocks

Request

```
{  
  "symbol": "GOOGL"  
}
```

Deny Esc

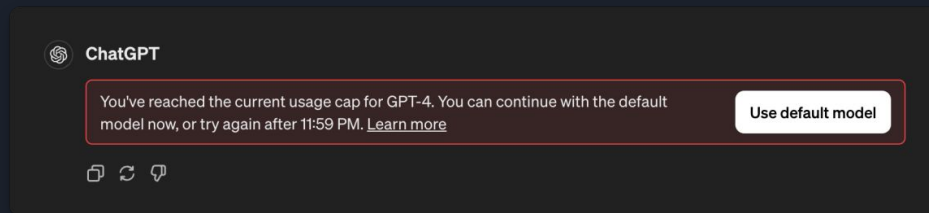
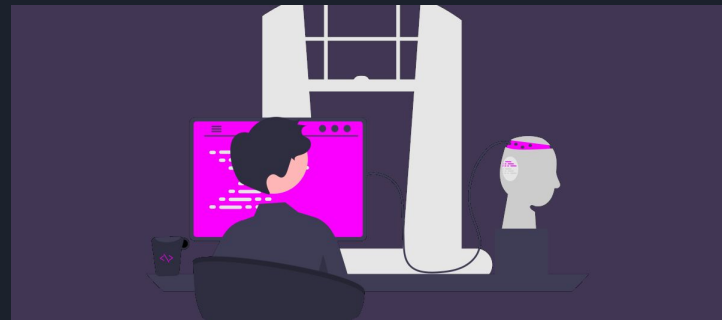
Always allow % ↵

Allow once ↵



Challenges

- Running out of tokens/prompts
 - Claude Pro
- High Response Latency
 - Switching to faster models





Thank you!

Questions?